CMPUT 461/561 Fall 2026

# Lec 06 -- Probabilistic Language Models

© Denilson Barbosa





## Outline

1. What is a Language Model
2. Estimating probabilities with N-grams
3. Evaluation and perplexity
4. Generating text with language models
5. Data sparsity
    - Unseen $n$-grams of previously seen tokens
    - OOV tokens
    - Discounting, back-off, interpolation



# Language modeling



## Models

A model is an informative representation of an object, person, or system.
https://en.wikipedia.org/wiki/Model

### Properties of a model

| | |
|---|---|
| **Mapping** | A model always is **a model of something**—it is an image or representation of some natural or artificial, existing or imagined original where this original itself could be a model. |
| **Reduction** | In general, a model will not include all attributes that describe the original but only those that appear relevant to the model's creator or user. |
| **Pragmatism** | A model [...] is intended to work as a replacement for the original. |

## Uses of a model

| | |
|---|---|
| **Prediction** | reliably anticipating unknown data, including data within the domain of the training data (interpolation), and outside the domain (extrapolation). |
| **Explanation** | establishing plausible chains of causality by proposing mechanisms that can explain patterns seen in data. |
| **Theoretical exposition** | discovering or proposing new hypotheses, or refuting existing hypotheses about the behaviour of the system being modelled. |
| **Description** | representing important aspects of the system being modelled. |
| **Illustration** | communicating an idea or explanation. |



# Models of language

A **language model** is a model of the human brain's ability to produce natural language.
https://en.wikipedia.org/wiki/Language_model

**Non-probabilistic** models
Typically, created by linguists.

Ex: **CFGs** can be used to predict, explain, describe and illustrate sentences that belong in the language.

**Probabilistic** models
Typically, learned from data.

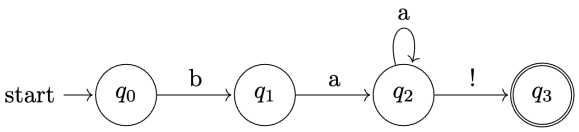
Ex: Probabilistic **CFGs**, $n$**-gram** language models, LLMs.

## Examples of non-probabilistic models

**Language**: sheep-talk




**Model**:




**Language**: integer arithmetic

```
(37 * 2) / 25
47 * 12 + 89
-156 / 4 - 23
```

...

**Model**:

```
E -> NUM | '(' E ')'
  | '-' E
  | E OP E

NUM -> '#NUM#'

OP -> '+' | '-' | '*' | '/'
```



## A probabilistic model of sheep-talk

**Data collection**: record the sheep.




**Sample data**:

`baaaa!`, `baaaaaa!`, `baa!`, `baaaaa!`,
`baaa!`, `baaaa!`, `baaaaa!`, `ba!`,
`baaaa!`, `baaaaaa!`, `baa!`, `baaaaa!`,
`baaaa!`, `baaa!`, `baaaaa!`, `baaaaaaa!`,
`baaa!`, `baaaa!`, `ba!`, `baaaaa!`

**Data modelling**: represent the data in a way that it is easy to estimate probabilities for each sentence from the sample.

## A probabilistic model of sheep-talk

A probabilistic model allows answering questions like:

**How likely is an event?**

Q: How often do sheep say `baaa!`?

A: P(`baaa!`)=0.15

**Is one event more likely than another?**

Q: Do sheep say `ba!` more often than `baaaa!`?

A: No, because P(`baaaa!`)>P(`ba!`)

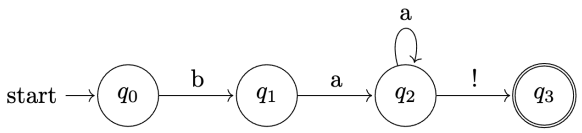
**Model**

| term | count | Probability |
|---|---|---|
| `ba!` | 2 | 2/20= 0.10 (10%) |
| `baa!` | 2 | 2/20= 0.10 (10%) |
| `baaa!` | 3 | 3/20= 0.15 (15%) |
| `baaaa!` | 5 | 5/20= 0.25 (25%) |
| `baaaaa!` | 5 | 5/20= 0.25 (25%) |
| `baaaaaa!` | 2 | 2/20= 0.10 (10%) |
| `baaaaaaa!` | 1 | 1/20= 0.05 (5%) |



## Probabilistic and linguistic models

**Model $M_1$**




**Model $M_2$**

$P(\texttt{ba!}) = 0.1$; $P(\texttt{baa!}) = 0.1$;
$P(\texttt{baaa!}) = 0.15$; $P(\texttt{baaaa!}) = 0.25$;
$P(\texttt{baaaaa!}) = 0.25$; $P(\texttt{baaaaaa!}) = 0.1$;
$P(\texttt{baaaaaa!}) = 0.05$

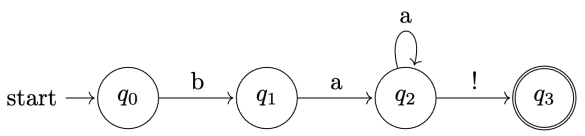
Questions that **both models can answer**:

- `baaa!` and `ba!` are sheep-talk while `booo!` isn't.

However, $M_1$ cannot answer questions about likelihoods (previous slide).

## Data sparsity and probabilistic models

**Model $M_1$**




**Model $M_2$**

$P(\texttt{ba!}) = 0.1$; $P(\texttt{baa!}) = 0.1$;
$P(\texttt{baaa!}) = 0.15$; $P(\texttt{baaaa!}) = 0.25$;
$P(\texttt{baaaaa!}) = 0.25$; $P(\texttt{baaaaaa!}) = 0.1$;
$P(\texttt{baaaaaa!}) = 0.05$

Are there questions $M_1$ can answer and $M_2$ cant?

**Yes**: $M_1$ can verify that `baaaaaaaaaaaaaaaaaa!` is sheep-talk.

This is an example of the **data sparsity** problem: the data used to learn $M_2$ did not contain that particular sheep-talk sentence.





# Estimating probabilities with $n$-grams

# $n$-gram Language Models

An **$n$-gram language model** is a statistical model of language which calculates the probability of the next **token** in a sequence from the previous **$n$-1 tokens**.

**Tokens can be**: whole words, normalized (whole) words, individual characters, or **sub-word** pieces (ex: syllables, prefixes + lemmas + suffixes, etc.).

### Special cases:

- **Unigram model**: $n = 1$; probabilities are estimate from 1-1=**0** previous tokens.
- **Bigram model**: $n = 2$; estimates use 2-1=**1** previous token.
- **Trigram model**: $n = 3$; estimates use 3-1=**2** previous tokens.



### Example unigram sentence-level model

$$M = \{P(\texttt{ba!}) = 0.1; \quad P(\texttt{baa!}) = 0.1; \quad P(\texttt{baaa!}) = 0.15; \quad P(\texttt{baaaa!}) = 0.25;$$
$$P(\texttt{baaaaa!}) = 0.25; \quad P(\texttt{baaaaaa!}) = 0.1; \quad P(\texttt{baaaaaa!}) = 0.05\}$$

Note that the sum of probabilities across all *events* in the model (the "sheep-sentences") must **add-up to one**.

Note also that, because the "sheep-sentences" are always a single string of characters, we cannot build word-level bigram or trigram models.

## Character level bigram model of sheep-talk

**Sample data**: `<s>baaaa!</s>`, `<s>baaaaaa!</s>`, `<s>baa!</s>`, `<s>baaaaa!</s>`, `<s>baaa!</s>`, `<s>baaaa!</s>`, `<s>baaaaa!</s>`, `<s>ba!</s>`, `<s>baaaa!</s>`, `<s>baaaaaa!</s>`, `<s>baa!</s>`, `<s>baaaaa!</s>`, `<s>baaaa!</s>`, `<s>baaa!</s>`, `<s>baaaaa!</s>`, `<s>baaaaaaa!</s>`, `<s>baaa!</s>`, `<s>baaaa!</s>`, `<s>ba!</s>`, `<s>baaaaa!</s>`

`<s>` and `</s>` are special symbols to indicate the start and the end of a sentence.

- We always add ($n-1$ of) them in $n$-gram models when $n > 1$.
  - With trigram models we add `<s><s>` and `</s></s>`
- This allows us to distinguish when a token appears at the start or in the middle of a sentence.



## Character level bigram model of sheep-talk

**Sample data:**

| bigram | count |
|---|---|
| `<s>b` | 20 |
| `ba` | 20 |
| `aa` | 59 |
| `a!` | 20 |
| `!</s>` | 20 |

**Model**:

$$P(\texttt{b}|\texttt{<s>}) = \frac{\text{count}(\texttt{<s>b})}{\text{count}(\texttt{<s>}\underline{*})} = \frac{20}{20} = 1$$

$$P(\texttt{a}|\texttt{b}) = \frac{\text{count}(\texttt{ba})}{\text{count}(\texttt{b}\underline{*})} = \frac{20}{20} = 1$$

$$P(\texttt{a}|\texttt{a}) = \frac{\text{count}(\texttt{aa})}{\text{count}(\texttt{a}\underline{*})} = \frac{59}{79} = 0.74$$

$$P(\texttt{!}|\texttt{a}) = \frac{\text{count}(\texttt{a!})}{\text{count}(\texttt{a}\underline{*})} = \frac{20}{79} = 0.26$$

$$P(\texttt{</s>}|\texttt{!}) = \frac{\text{count}(\texttt{!</s>})}{\text{count}(\texttt{!}\underline{*})} = \frac{20}{20} = 1$$

## Notation of our bigram model

$$P(\texttt{b}|\texttt{<s>}) = \frac{\text{count}(\texttt{<s>b})}{\text{count}(\texttt{<s>}\,\underline{*}\,)} = \frac{20}{20} = 1$$

- $P(\texttt{b}|\texttt{<s>})$ ≡ the probability with which a `b` follows a `<s>`.
- $\text{count}(\texttt{<s>b})$ ≡ the number of times bigram `<s>b` appears.
- $\text{count}(\texttt{<s>}\,\underline{*}\,)$ ≡ count of all bigrams starting with `<s>`.

$$\text{count}(w_i\,\underline{*}\,) = \sum_{w_j \in V} \text{count}(w_i\,w_j)$$

where $V$ is the **vocabulary** (set of all tokens in the model).



## Example inferences

**Model**:

$$P(\texttt{b}|\texttt{<s>}) = 1; \quad P(\texttt{a}|\texttt{b}) = 1; \quad P(\texttt{a}|\texttt{a}) = 0.74; \quad P(\texttt{!}|\texttt{a}) = 0.26; \quad P(\texttt{</s>}|\texttt{!}) = 1$$

Probability of `baa!`?

$$P(\texttt{b}|\texttt{<s>}) \cdot P(\texttt{a}|\texttt{b}) \cdot P(\texttt{a}|\texttt{a}) \cdot P(\texttt{!}|\texttt{a}) \cdot P(\texttt{</s>}|\texttt{!})$$
$$= 1 \cdot 1 \cdot 0.74 \cdot 0.26 \cdot 1 = 0.1924$$

Probability of `ba!`?

$$P(\texttt{b}|\texttt{<s>}) \cdot P(\texttt{a}|\texttt{b}) \cdot P(\texttt{!}|\texttt{a}) \cdot P(\texttt{</s>}|\texttt{!})$$
$$= 1 \cdot 1 \cdot 0.26 \cdot 1 = 0.26$$

Recall the estimates with the **unigram model**? P(`baa!`)=0.1 and P(`ba!`)=0.05

- What gives?

**Model**:

$$P(\texttt{b}|\texttt{<s>}) = 1; \quad P(\texttt{a}|\texttt{b}) = 1; \quad P(\texttt{a}|\texttt{a}) = 0.74; \quad P(\texttt{!}|\texttt{a}) = 0.26; \quad P(\texttt{</s>}|\texttt{!}) = 1$$

Probability of `boo!` ≡ $P(\texttt{b}|\texttt{<s>}) \cdot P(\texttt{o}|\texttt{b}) \cdot P(\texttt{o}|\texttt{o}) \cdot P(\texttt{!}|\texttt{o}) \cdot P(\texttt{!}|\texttt{</s>})$?

- **Zero**, because $P(\texttt{o}|\texttt{b}) = P(\texttt{o}|\texttt{o}) = P(\texttt{!}|\texttt{o}) = 0$ according to the model

Note that the bigram model **can** estimate the probability of `baaaaaaaaaaa!`, which the unigram model we saw before couldn't!

- In fact, it can estimate the probability of sheep-talk with any sequence of `a`s!
- Also, the longer the sequence, the lower the probability





# The theory behind $n$-gram estimates

# Probabilistic Language Modeling

**Goal**: estimate the probability of a sentence (i.e., sequence of tokens).

$$P(W) = P(w_1, w_2, w_3, w_4, w_5, \ldots, w_k)$$

From the **definition of conditional probabilities**: $P(x, y) = P(x) \cdot P(y|x)$

Using the **chain rule** over $k$ variables:

$$P(w_1, w_2, w_3, \ldots, w_k) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \cdots P(w_k|w_1, \ldots, w_{k-1})$$



## Example

$$P(\texttt{its}, \texttt{water}, \texttt{is}, \texttt{so}, \texttt{clear}, \texttt{that}) =$$
$$P(\texttt{its})P(\texttt{water}|\text{its})P(\texttt{is}|\texttt{its}, \texttt{water}) \cdots P(\text{that}|\texttt{its}, \texttt{water}, \texttt{is}, \texttt{so}, \texttt{clear})$$

Estimating the **joint probability** of a sequence of tokens using the chain rule:

$$P(w_1, \ldots, w_k) = \prod_i^k P(w_i|w_1, \ldots, w_{i-1})$$

## Estimating probabilities

Can't we just count, like we did with bigrams?

$$P(\text{that}|\texttt{its}, \texttt{water}, \texttt{is}, \texttt{so}, \texttt{clear}) = \frac{\text{count}(\texttt{its water is so clear that})}{\text{count}(\texttt{its water is so clear } \underline{*})}$$

### Data sparsity strikes again

**Counting isn't feasible**. There are just too many possible sentences and we won't ever get enough data to estimate counts for all of them.

Even with access to Web-scale corpora, we can easily write a new sentence that nobody has written before!



# Simplifying assumption

Inspired by the **Markov assumption** (stochastic processes are memoryless), we can approximate probabilities by assuming that the $i$-th token in a sequence depends on just a few of the previous ones (instead of all $i-1$).

| model | assumption | context |
|---|---|---|
| 1-gram (**unigram**) | $P(w_i\|w_1,\ldots,w_{i-1}) \approx P(w_i)$ | 0 tokens |
| 2-gram (**bigram**) | $P(w_i\|w_1,\ldots,w_{i-1}) \approx P(w_i\|w_{i-1})$ | 1 tokens |
| 3-gram (**trigram**) | $P(w_i\|w_1,\ldots,w_{i-1}) \approx P(w_i\|w_{n-2},w_{n-1})$ | 2 tokens |
| 4-gram | $P(w_i\|w_1,\ldots,w_{i-1}) \approx P(w_i\|w_{i-3},w_{i-2},w_{i-1})$ | 3 tokens |
| $n$-gram | $P(w_i\|w_1,\ldots,w_{i-1}) \approx P(w_i\|w_{i-n-1},w_{i-n-2},\ldots,w_{i-1})$ | $n-1$ tokens |

## Estimating probabilities (under the Markov assumption)

Estimating the **joint probability** of a sequence of tokens using the chain rule:

$$P(w_1, \ldots, w_k) = \prod_i^k P(w_i | w_1, \ldots, w_{i-1})$$

Estimating the **joint probability** of a sequence of tokens using $n$-**grams**:

$$P(w_1, \ldots, w_k) = \prod_i^k P(w_i | w_{i-n-1}, \ldots, w_{i-1})$$



### Example:

**Trigram** estimate:

$$P(\text{that} | \texttt{its}, \texttt{water}, \texttt{is}, \texttt{so}, \texttt{clear}) \approx P(\text{that} | \texttt{so}, \texttt{clear}) = \frac{\text{count}(\texttt{so clear that})}{\text{count}(\texttt{so clear } \underline{*})}$$

**Bigram** estimate:

$$P(\text{that} | \texttt{its}, \texttt{water}, \texttt{is}, \texttt{so}, \texttt{clear}) \approx P(\text{that} | \texttt{clear}) = \frac{\text{count}(\texttt{clear that})}{\text{count}(\texttt{clear } \underline{*})}$$

**Unigram** estimate:

$$P(\text{that} | \texttt{its}, \texttt{water}, \texttt{is}, \texttt{so}, \texttt{clear}) \approx P(\text{that}) = \frac{\text{count}(\texttt{that})}{\text{count}(\underline{*})}$$

### Data sparsity and $n$

Generally, the longer the sequence, the harder it becomes to obtain reliable counts, even if the sample data is as large as the Web.

There are many reasons for that.

For example, **typos** and other errors accumulate independently of one another. The longer the sequence, the higher the chance that at least one token has an error.

**Trigram** estimate:

$$P(\text{that}|\texttt{so}, \texttt{clear}) = \frac{\text{count}(\texttt{so clear that})}{\text{count}(\texttt{so clear } \underline{*})}$$

**Bigram** estimate:

$$P(\text{that}|\texttt{clear}) = \frac{\text{count}(\texttt{clear that})}{\text{count}(\texttt{clear } \underline{*})}$$

**Unigram** estimate:

$$P(\text{that}) = \frac{\text{count}(\texttt{that})}{\text{count}(\underline{*})}$$





## What do $n$-grams learn?

Data from the Berkeley Restaurant Project (BeRP) Transcripts

- The BeRP system was designed to be an automated consultant whose domain of knowledge was restaurants in the city of Berkeley.
- The BeRP corpus, collected at ICSI, currently contains approximately 8566 utterances, with about 1900 unique words, comprising about 7 hours of speech.

https://github.com/wooters/berp-trans

## BeRP Transcripts excerpt

berp-trans / transcript.txt

Code | Blame | 8566 lines (8566 loc) · 380 KB

```
95   13_1_0001 [lip_smack] i was thinking about going out for some french food tonight
96   13_1_0002 how about [loud_breath] twelve dollars a person
97   13_1_0003 i'll try to get a car so it shouldn't matter [loud_breath]
98   13_1_0004 [loud_breath] why don't you make that up to sixteen dollars
99   13_1_0005 [loud_breath] can i have more information on [um] [lip_smack] metropole
100  13_1_0006 can you tell me about . asian food in berkeley
101  13_1_0007 dinner any night
102  13_1_0008 i'm a student so it had best not be better than more more [laughter] best not be mo
103  13_1_0009 [lip_smack] doesn't matter how far i wanna see everything here
104  13_1_0010 can you tell me more about oriental food to go <to> <go> and [um] vietnam village
105  13_1_0011 where's the most expensive russian place in berkeley
106  13_1_0012 [loud_breath] dinner is best
107  13_1_0013 it can be anywhere
108  13_1_0014 [lip_smack] a-la-carte . addison annex . amaru . arinell pizza . augusta's
109  13_1_0015 ay-caramba . berkeley thai house . bette's ocean view . blondie's pizza . blue nile
110  13_1_0016 brennan's . brick hut cafe . brit-marie's . cactus taqueria . cafe fanny
111  13_1_0017 cafe mar-mara . cafe pastoral . cafe vin . cafe violeta . caffe giovanni
112  13_1_0018 caffe venezia . cambodiana's . casa-de-eva . casbah . cha-am
113  13_1_0019 chez-panisse . chez-panisse cafe . christopher's cafe . claremont seafood . coffee
114  13_1_0020 continental garden . daruma teashop . elmwood pharmacy . everett and jones barbecue
115  13_1_0021 fat apple's . flint's barbecue . fourth street grill . gertie's . gramma's inn
```



## Some bigram counts from the BeRP data

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| **i** | 5 | 827 | 0 | 9 | 0 | 0 | 0 | 2 |
| **want** | 2 | 0 | 608 | 1 | 6 | 6 | 5 | 1 |
| **to** | 2 | 0 | 4 | 686 | 2 | 0 | 6 | 211 |
| **eat** | 0 | 0 | 2 | 0 | 16 | 2 | 42 | 0 |
| **chinese** | 1 | 0 | 0 | 0 | 0 | 82 | 1 | 0 |
| **food** | 15 | 0 | 15 | 0 | 1 | 4 | 0 | 0 |
| **lunch** | 2 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| **spend** | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |

**Figure 3.1** Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray. Each cell shows the count of the column label word following the row label word. Thus the cell in row **i** and column **want** means that **want** followed **i** 827 times in the corpus.

### The highest-count bigrams in the table are grammatical

Note that all bigrams with counts over 40 in the previous table contain grammatical constituents!

Some bigrams represent "partial" constituents:
**Ex:** "eat Chinese " (probably before "food"?)

Zero-count bigrams are ungrammatical!
**Ex:** "I food", "food lunch", "eat want",...

| bigram | POS | grammatical? |
|---|---|---|
| I want | PRP VB | ✅ |
| to eat | TO VB | ✅ |
| want to | VB RP | ✅ |
| to spend | TO VB | ✅ |
| Chinese food | JJ NN | ✅ |
| eat lunch | VB NN | ✅ |



## Can $n$-gram language models learn grammar?

Fundamentally, **probabilistic language models learn word associations from the training data**.

Feed them grammatical text and they will "learn grammar".

- More precisely, they will learn to associate words appearing in grammatical sentences.

However, it is not easy to "decode" a probabilistic model to extract grammatical knowledge!

Language models like ELMo, GPT-2, and BERT, spawned the study of "BERTology", which attempts to interpret what is learned by these models. **BERT**:

① has hierarchical representations ("capturing" constituents).
② encodes PoS information.
③ "does" subject-predicate agreement.
④ does not "understand" negation.

https://aclanthology.org/2020.tacl-1.54/

## What else do $n$-gram models learn?

The BeRP bigrams tell us that $P(\texttt{I}, \texttt{want}, \texttt{chinese}, \texttt{food}) > P(\texttt{I}, \texttt{want}, \texttt{english}, \texttt{food})$, because $P(\texttt{chinese}|\texttt{want})$ is six times higher than $P(\texttt{english}|\texttt{want})$ **in that corpus**.

### What does this mean?

① Chinese food is better than English food?
② Everyone prefers Chinese food over English food?
③ CS students living in Berkeley prefer Chinese food over English food?

Probabilistic language models **cannot** answer that question!



# $n$-gram language models and long sentences

PLMs cannot feasibly represent **long-distance dependencies**:

**The computer** which I had just put into the machine room on the fifth floor **crashed**.

Practical $n$-gram models ($n \leq 5$) would not learn to associate "crashing" to computers from that sentence.

Therefore, they would be **extremely unlikely** to generate a sentence like that.

PS: LLMs solved that problem.



## ASIDE: avoiding numerical underflow

Estimating probability of long sequences of tokens using $n$-grams requires multiplying many numbers between 0 and 1, which can lead to **numerical underflow**:

$$P(w_1, \ldots, w_k) = \prod_i^k P(w_i | w_{i-n-1}, \ldots, w_{i-1})$$

Instead, we often operate in the world of **log probabilities**:

$$\hat{P}(w_1, \ldots, w_k) = \sum_i^k \log P(w_i | w_{i-n-1}, \ldots, w_{i-1})$$

While these are not numerically identical, if $P(S_1) > P(S_2)$ then $\hat{P}(S_1) > \hat{P}(S_2)$.





## ASIDE: toolkits

There are many libraries for building and using probabilistic language models out there.

Some examples:

1. CMU -- https://www.cs.cmu.edu/~dorcas/toolkit_documentation.html
2. SRI -- http://www.speech.sri.com/projects/srilm/
3. KenLM -- https://kheafield.com/code/kenlm/
4. KenLM -- https://github.com/kpu/kenlm
5. NLTK -- https://www.nltk.org/api/nltk.lm.html
6. Spacy -- https://pypi.org/project/spacy-ngram/



## ASIDE: plenty of $n$-gram data available

In 2006, Google released $n$-grams from Web pages, collected to help with machine translation. The corpus has about one *trillion* tokens, and contains up to **5-grams**.

**Update (22 Sept. 2006):** The LDC now has the data available in their catalog. The counts are as follows:

```
File sizes: approx. 24 GB compressed (gzip'ed) text files

Number of tokens:    1,024,908,267,229
Number of sentences:    95,119,665,584
Number of unigrams:         13,588,391
Number of bigrams:         314,843,401
Number of trigrams:        977,069,902
Number of fourgrams:     1,313,818,354
Number of fivegrams:     1,176,470,663
```

https://research.google/blog/all-our-n-gram-are-belong-to-you/





## Model evaluation and perplexity

- Extrinsic vs intrinsic evaluations
- Perplexity

# Evaluating an $n$-gram model

Does the language model prefer “good” sentences to “bad” ones?

- Good and bad could relate to grammaticality, common language patterns, or other criteria (ex: culinary knowledge).
- Make sure your **training data reflects whatever you mean by good and bad**.

Nevertheless, a good model should assign higher probability to data that is consistent with the training data than it does to other data.

**Evaluation**

- We learn model parameters on a **training set**.
- We test the model’s performance on a **test set** (data the model hasn’t seen).
- We use an **evaluation metric** to tell us how well our model does on the test set.



# Extrinsic model evaluation

**Best evaluation** for comparing models $A$ and $B$.

Use each model in a task **that is important to you**:

- Ex: spelling correction, speech recognition, machine translation

Run multiple tasks and get an accuracy for $A$ and for $B$.

- How many misspelled words corrected properly...
- How many words translated correctly...

Compare **accuracy** or other measure(s) of performance for $A$ and $B$.

## Extrinsic vs intrinsic evaluations

**Extrinsic evaluations are time-consuming**: the can take days or weeks, especially if we want to run them on live production systems.

**Extrinsic evaluations are expensive**: we need to set-up (and test) systems, recruit users, monitor user performance, study the data, ...

**Extrinsic evaluations can damage reputation**: testing any kind of new feature on live systems can lead to PR disasters.

**Intrinsic evaluations** of $n$-gram models:

Evaluate model on **test data** that is similar to what you trained with.

Measure: **perplexity**.

Bad approximation for "truly unseen" data that users in a live system would use.

Generally, only useful as a preliminary evaluation (ex: proof-of-concept or in academic publications).



# Perplexity

Perplexity is the **inverse probability of the test set**, normalized by the number of words.

$$PP(W = w_1, w_2, \ldots, w_N) = P(W)^{-\frac{1}{N}}$$

Using the chain rule and $n$-gram estimates:

$$PP(W) = \sqrt[N]{\frac{1}{\prod_i^N P(w_i|w_{N-n-1}, \ldots, w_{N-1})}}$$

**Unigrams**:

$$PP(W) = \sqrt[N]{\frac{1}{\prod_i^N P(w_i)}}$$

**Bigrams**:

$$PP(W) = \sqrt[N]{\frac{1}{\prod_i^N P(w_i|w_{i-1})}}$$

**Trigrams**:

$$PP(W) = \sqrt[N]{\frac{1}{\prod_i^N P(w_i|w_{i-2}, w_{i-1})}}$$

# Better models have lower perplexity

Experiment with Wall Street Journal data:

- 38 million words in training data.
- 1.5 million words in test data.

| | Unigram | Bigram | Trigram |
|---|---|---|---|
| Perplexity | 962 | 170 | 109 |

Note: the **same** training and test data were used for all three models.



## Why do we take the $N$-th root when computing perplexity?

Taking the $N$-th root gives us the **geometric mean** of the inverse probabilities.

**Normalizing by sequence length**, allows us to meaningfully compare perplexity across different texts.

- A model with perplexity 50 is equally uncertain per token whether the sequence is 10 or 1000 tokens long.

If we just used the raw product of probabilities, longer sequences would naturally have lower probabilities simply because we're multiplying more terms less than one . This would make perplexity incomparable across sequences of different lengths.



## ASIDE: Perplexity as "average branching factor"

Within **Information Theory**, perplexity is interpreted as answering: "On average, how many choices does the model have at each position?"

### Example

Imagine a language whose sentences are sequences of digits (0-9), and assume a unigram model that assigns the same probability 1/10 to all digits.

What is $PP(W = w_1, \ldots, w_N)$ according to this model?

$$PP(W) = \sqrt[N]{\frac{1}{P(w_1, \ldots, w_N)}} = \sqrt[N]{\frac{1}{(1/10)^N}} = \sqrt[N]{10^N} = 10$$

Regardless of sequence length, the model has 10 choices per digit.



## ASIDE: Perplexity as "average branching factor"

### Example #2:

Let $W$ be 120K phone calls each asking for one employee (out of 30,000) at random.

What is the perplexity of this sequence?

$$PP(W) = \sqrt[120K]{\frac{1}{\left(\frac{1}{30K}\right)^{120K}}} = \sqrt[120K]{(30K)^{120K}} = 30K$$

That is, whoever is guessing has 30K "choices" for the next token.



## ASIDE: Perplexity as "average branching factor"

**Example #3:**

Consider 120K phone calls as follows: 75% of them request a transfer to a department (out of four: sales, research, production, warranty) with equal probability; 25% ask for an employee (out of 30,000) at random.

What is the perplexity of this data?

$$PP(W) = \sqrt[120K]{\frac{1}{(1/4)^{90K} \cdot (1/30K)^{30K}}} = \sqrt[120K]{4^{90K} \cdot (30K)^{30K}}$$

$$PP(W) = \sqrt[120K]{(4^3 \cdot 30K)^{30K}} = \sqrt[4]{4^3 \cdot 30K} \approx 37.2$$

Guessing is easier now, because there are fewer choices.

**This is an easier language to learn.**





## Generating text from $n$-gram language models

# Natural Language Generation

**Natural Language Generation** (NLG) is the task of generating natural language from structure data (e.g., a language model).

NLG is a key component in many applications today, including in industry, for example, as dialogue systems to act as UIs for data-intensive applications.

ELIZA's approach to NLG was to rely on rules for match-and-replace words using regular expressions.

$n$-gram language models play an important role in NLG: We can generate language by **sampling from the model**.



## Monte Carlo Sampling for NLG

(Deoras et al., 2013)

**Main idea** (using bigrams, for simplicity):

1. Starting from `<s>`, sample the next token following the probability distribution conditioned on `<s>`, call it $w_1$.
2. Next, sample the next token from the probability distribution conditioned on $w_1$.
3. Continue sampling conditioned on the previous token until `</s>` is sampled or the algorithm went over too many steps (and, if so, restart).

**Optional**: generate a number of sentences first, and return the one with the **least perplexity**.

## Example

**Step 1**:
$w_1 =$ `Not`, because that's the only event in the distribution conditioned on `<s>` ($P(\texttt{Not}|\texttt{<s>}) = 1$)

**Step 2**:
$w_2$ can be either `in` or `with` and both are equally likely.

**How do we choose?**

```
<s> Not in a box </s>
<s> Not with a fox </s>
<s> Not in a house </s>
<s> Not with a mouse </s>
```

**Model:**

$P(\texttt{Not}|\texttt{<s>}) = 1; P(\texttt{in}|\texttt{Not}) = P(\texttt{with}|\texttt{Not}) = 0.5;$

$P(\texttt{a}|\texttt{in}) = P(\texttt{a}|\texttt{with}) = 1;$

$P(\texttt{box}|\texttt{a}) = P(\texttt{fox}|\texttt{a}) = 0.25;$

$P(\texttt{house}|\texttt{a}) = P(\texttt{mouse}|\texttt{a}) = 0.25;$



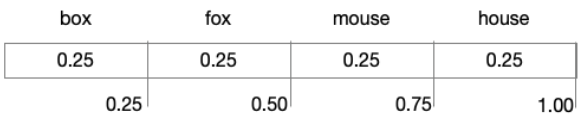
## Choosing the next token

When generating token $w_i$, after generating $h = w_1, \ldots, w_{i-1}$, we query the language model to obtain a probability distribution, $P(w_j|h)$, over all the tokens in the vocabulary.

We can ignore tokens such that $P(w_j|h) = 0$. (NOTE: LLMs discard all tokens whose $P(w_j|h)$ is below a threshold).

Bin all candidates together, forming a cumulative distribution (from 0 to 1).

**Example**: tokens that can come after `a` in our bigram model.




Finally, generate a **random number in (0,1)** and pick the word corresponding to that number!

- If the number is 0.6732, pick `mouse`; if it is 0.1, pick `box`, and so on.

# Example

**Unseen sentences** that can be generated:

`<s> Not with a box </s>`

`<s> Not in a fox </s>`

`<s> Not with a house </s>`

This is a very simple model, as we only have to sample after `Not` (two choices) and after `a` (four).

Also, all sentences have the same length.

```
<s> Not in a box </s>
<s> Not with a fox </s>
<s> Not in a house </s>
<s> Not with a mouse </s>
```

**Model:**

$P(\texttt{Not}|\texttt{<s>}) = 1; P(\texttt{in}|\texttt{Not}) = P(\texttt{with}|\texttt{Not}) = 0.5;$

$P(\texttt{a}|\texttt{in}) = P(\texttt{a}|\texttt{with}) = 1;$

$P(\texttt{box}|\texttt{a}) = P(\texttt{fox}|\texttt{a}) = 0.25;$

$P(\texttt{house}|\texttt{a}) = P(\texttt{mouse}|\texttt{a}) = 0.25;$



## Another example

**Unseen sentences** that are possible:

`<s> I am </s>`

`<s> I am Sam I am </s>`

`<s> Sam I do not like green eggs and ham </s>`

This model allows "infinite loops":

`<s> I am Sam I am Sam I am Sam ...`

This is why we need to **stop the generation** if `</s>` is not reached within a finite number of steps.

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I do not like green
    eggs and ham </s>
```

Sampling choices:

$P(\underline{*}|\texttt{<s>}) =$

| I | Sam |
|---|---|
| 2/3 | 1/3 |

$P(\underline{*}|\texttt{I}) =$

| do | am |
|---|---|
| 1/3 | 2/3 |

$P(\underline{*}|\texttt{am}) =$

| </s> | Sam |
|---|---|
| 1/2 | 1/2 |

$P(\underline{*}|\texttt{Sam}) =$

| </s> | I |
|---|---|
| 1/2 | 1/2 |



## Data sparsity

Unseen $n$-grams of previously seen terms
Out-of-Vocabulary (OOV) terms
Addressing data sparsity

- Discounting
- Back-off
- Interpolation
- Corpus-level smoothing
- Out-of-Vocabulary terms



## Data sparsity and zeroes

**Data sparsity** refers to the presence of gaps, missing values, or irregular sampling intervals in training data used to build the language model.

Training data:

```
… denied the allegations
… denied the claims
… denied the request
… accepted the offer
```

Test data:

```
… denied the offer
… denied the loan
```

This model would estimate $P(\texttt{offer}|\texttt{denied}, \texttt{the}) = 0$, and that is bad!

- Although missing from the training data, "denied the offer" is good language.

## Zero probability $n$-grams of previously seen tokens

The $n$-gram `denied the offer` is made of **tokens that appear in the training data**, but not in this particular order!

$n$-grams with zero probability:

- Assign a 0 probability to the entire test set.
- Don't allow us to compute perplexity because we can't divide by 0.

A different problem, which we will see later, is dealing with tokens that do not appear in the training data (Out-of-Vocabulary, or OOV).

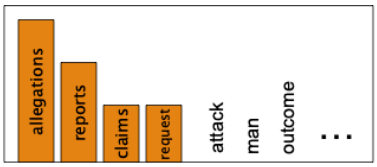


# Discounting (a.k.a., smoothing) -- intuition

Model distributions will have zeroes whenever $n$-grams have zero counts.

Ex: $\text{count}(\texttt{denied the attack}) = 0$

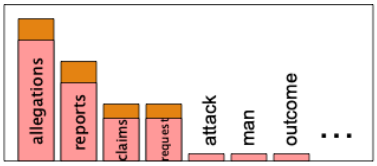
Original $P(\underline{*}|\texttt{denied}, \texttt{the})$:




Solution: "steal" probability mass from the events that were observed in the training data and "give" them to events that were not seen.

If not correctly, the result is a different but proper probability distribution.

Smoothed $P(\underline{*}|\texttt{denied}, \texttt{the})$:



# Add-one (Laplace) smoothing

**Assumption**: counts of both observed and unobserved events are off, by one.

Illustration with bigrams:

Maximum-Likelihood Estimate:

$$P(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}\, w_i)}{\text{count}(w_{i-1}\, \underline{*})}$$

Add-1 estimate:

$$\hat{P}(w_i|w_{i-1}) = \frac{\text{count}(w_{i-1}\, w_i) + 1}{\sum_{w_j \in V} \text{count}(w_{i-1}\, w_j) + 1}$$

$$\hat{P}(w_i|w_{i-1}) = \frac{\text{count}(w_i, w_{i-1}) + 1}{\text{count}(w_{i-1}\, \underline{*}) + |V|}$$

**Generalization**: add-$k$ smoothing, in which we add $k$ (instead of 1) to every count.



## Example: BeRP data

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| **i** | 5 | 827 | 0 | 9 | 0 | 0 | 0 | 2 |
| **want** | 2 | 0 | 608 | 1 | 6 | 6 | 5 | 1 |
| **to** | 2 | 0 | 4 | 686 | 2 | 0 | 6 | 211 |
| **eat** | 0 | 0 | 2 | 0 | 16 | 2 | 42 | 0 |
| **chinese** | 1 | 0 | 0 | 0 | 0 | 82 | 1 | 0 |
| **food** | 15 | 0 | 15 | 0 | 1 | 4 | 0 | 0 |
| **lunch** | 2 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| **spend** | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |

**Figure 3.1** Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Zero counts are in gray. Each cell shows the count of the column label word following the row label word. Thus the cell in row **i** and column **want** means that **want** followed **i** 827 times in the corpus.

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| **i** | 0.002 | 0.33 | 0 | 0.0036 | 0 | 0 | 0 | 0.00079 |
| **want** | 0.0022 | 0 | 0.66 | 0.0011 | 0.0065 | 0.0065 | 0.0054 | 0.0011 |
| **to** | 0.00083 | 0 | 0.0017 | 0.28 | 0.00083 | 0 | 0.0025 | 0.087 |
| **eat** | 0 | 0 | 0.0027 | 0 | 0.021 | 0.0027 | 0.056 | 0 |
| **chinese** | 0.0063 | 0 | 0 | 0 | 0 | 0.52 | 0.0063 | 0 |
| **food** | 0.014 | 0 | 0.014 | 0 | 0.00092 | 0.0037 | 0 | 0 |
| **lunch** | 0.0059 | 0 | 0 | 0 | 0 | 0.0029 | 0 | 0 |
| **spend** | 0.0036 | 0 | 0.0036 | 0 | 0 | 0 | 0 | 0 |

**Figure 3.2** Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences. Zero probabilities are in gray.

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| **i** | 6 | 828 | 1 | 10 | 1 | 1 | 1 | 3 |
| **want** | 3 | 1 | 609 | 2 | 7 | 7 | 6 | 2 |
| **to** | 3 | 1 | 5 | 687 | 3 | 1 | 7 | 212 |
| **eat** | 1 | 1 | 3 | 1 | 17 | 3 | 43 | 1 |
| **chinese** | 2 | 1 | 1 | 1 | 1 | 83 | 2 | 1 |
| **food** | 16 | 1 | 16 | 1 | 2 | 5 | 1 | 1 |
| **lunch** | 3 | 1 | 1 | 1 | 1 | 2 | 1 | 1 |
| **spend** | 2 | 1 | 2 | 1 | 1 | 1 | 1 | 1 |

**Figure 3.6** Add-one smoothed bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences. Previously-zero counts are in gray.

| | i | want | to | eat | chinese | food | lunch | spend |
|---|---|---|---|---|---|---|---|---|
| i | 0.0015 | 0.21 | 0.00025 | 0.0025 | 0.00025 | 0.00025 | 0.00025 | 0.00075 |
| want | 0.0013 | 0.00042 | 0.26 | 0.00084 | 0.0029 | 0.0029 | 0.0025 | 0.00084 |
| to | 0.00078 | 0.00026 | 0.0013 | 0.18 | 0.00078 | 0.00026 | 0.0018 | 0.055 |
| eat | 0.00046 | 0.00046 | 0.0014 | 0.00046 | 0.0078 | 0.0014 | 0.02 | 0.00046 |
| chinese | 0.0012 | 0.00062 | 0.00062 | 0.00062 | 0.00062 | 0.052 | 0.0012 | 0.00062 |
| food | 0.0063 | 0.00039 | 0.0063 | 0.00039 | 0.00079 | 0.002 | 0.00039 | 0.00039 |
| lunch | 0.0017 | 0.00056 | 0.00056 | 0.00056 | 0.00056 | 0.0011 | 0.00056 | 0.00056 |
| spend | 0.0012 | 0.00058 | 0.0012 | 0.00058 | 0.00058 | 0.00058 | 0.00058 | 0.00058 |

**Figure 3.7** Add-one smoothed bigram probabilities for eight of the words (out of $V = 1446$) in the BeRP corpus of 9332 sentences computed by Eq. 3.26. Previously-zero probabilities are in gray.

# Smoothing ≡ discounting

Add-one smoothing is not ideal for language modelling.

- Experiments have shown that they lead to significant distortions of the probability distributions.
- Add-one smoothing is actually OK for Naive-Bayes classification though.

There are many more other forms of smoothing.

- Jelinek-Mercer and Kneser-Ney are two well-known.

When done correctly, **discounting always results in a proper distribution**.



# Back-off and interpolation

Recall that our problem is a zero caused by an $n$**-gram that was not seen in training**, even though all tokens were.

Ex: `denied the offer` was unseen in training, but `denied the` and `offer` were.

Training data:

```
… denied the allegations
… accepted the offer
```

Test data:

```
… denied the offer
```

Back-off and interpolation combine evidence from multiple models (ex: trigram, bigram, and unigram estimates). Because all tokens were seen (individually) in training, we know that at least the unigram counts are non-zero.

- **Back-off**: if $n$-gram estimate is zero, use $(n-1)$ estimates (recursively).
- **Interpolation**: combine estimates from $n$-grams, $(n-1)$-grams, .... (recursively).

# Interpolation

Illustrating the idea with $n = 3$.

$$\hat{P}(w_i|w_{i-2}\,w_{i-1}) = \lambda_1 P(w_i|w_{i-2}\,w_{i-1}) + \lambda_2 P(w_i|w_{i-2}) + \lambda_3 P(w_i)$$

$$\sum_i \lambda_i = 1$$

We keep trigram, bigram, and unigram models and we **always** use all three models when computing any estimate.

Recall that unigram counts cannot be zero in this case.

The (simple) interpolation above uses "global" weights ($\lambda_i$) but other approaches are possible, where per-term weights are used.

- This leads to significant;y more model parameters.



## Learning interpolation weights

For this to work we need to split the original data into three (as opposed to two sets): training, held-out, and test data.




Algorithm:

1. First, fix the $n$-gram probabilities using the training data.
2. Then search for $\lambda$s that yield lowest perplexity on the held-out data.
3. Finally, evaluate the model on the test data.

# Back-off

Use the estimate from the "highest $n$" available.

**Example**: suppose you have trigrams, bigrams and unigrams and want to estimate the probability of a trigram $w_1\ w_2\ w_3$. Then, you use:

- $P(w_3|w_1, w_2)$ if the trigram is present in the training corpus; otherwise you use
- $P(w_3|w_2)$ if the bigram is present in the training corpus; otherwise you use
- $P(w3)$ from the unigram counts.

Note that this works even when the sequence $w_1\ w_2\ w_3$ is never seen in training (as long as each $w_i$ is seen!).



## Example

Corpus:

```
the cat sat on the mat
the dog sat on the log
the cat ate
```

**Unigrams (N = 15)**

| word | count | P(w) |
|---|---|---|
| the | 5 | 5/15 = 0.333 |
| cat | 2 | 2/15 = 0.133 |
| sat | 2 | 2/15 = 0.133 |
| on | 2 | 2/15 = 0.133 |
| mat, dog, log, ate | 1x4 =4 | 1/15 = 0.067 |
| **total** | **15** | **1.000** |

## Example

Corpus:

```
the cat sat on the mat
the dog sat on the log
the cat ate
```

**Bigrams "$\text{the}\ \underline{*}$" (count(the) = 5)**

| bigram | count | P(w \| the) |
|---|---|---|
| the cat | 2 | 2/5 = 0.40 |
| the mat | 1 | 1/5 = 0.20 |
| the dog | 1 | 1/5 = 0.20 |
| the log | 1 | 1/5 = 0.20 |
| the sat, the the | 0 | 0 |
| **total** | **5** | **1.00** |



## $P(\text{the}\ w)$ =?

**Interpolation ($\lambda_2 = 0.7$, $\lambda_1 = 0.3$)**

| query | bigram part | unigram part | result |
|---|---|---|---|
| P(the cat) | 0.40 | 0.133 | 0.7 × 0.40 + 0.3 × 0.133 = **0.32** |
| P(the sat) | 0 | 0.133 | 0.7 × 0 + 0.3 × 0.133 = **0.04** |

**Back-off** (use the bigram if count > 0; otherwise use the unigram).

| query | count(the, w) | estimate used | result |
|---|---|---|---|
| P(the cat) | 2 > 0 | P(cat \| the): bigram only | **0.40** |
| P(the sat) | 0 | P(sat): unigram only | **0.133** |

## Back-off, interpolation, and discounting

All three deal with estimating the probability of n-grams that have never seen in training **but** are made of tkens that have all been seen in training.

Unlike with discounting and interpolation, back-off does not result in a **proper probability distribution**.

- For example, mixing unigram and trigram estimates may not add-up to 1.

There are other forms of back-off that provide better (and faster) estimates for real-life applications. (See the book).





## Zeroes caused by OOV (out-of-vocabulary) terms

# Out-Of-Vocabulary (OOV) tokens and zeroes

There is another source of zeroes, which has not been addressed so far: what if a token in the test data was never seen in training?

- It could be a typo, a rare proper noun, some made-up word, ...

Whatever the case, a token never seen in training is called **Out-Of-Vocabulary (OOV)**.

All OOV terms have very low (true) probability.

- Why? Otherwise they would be seen in training!



## Dealing with OOV tokens

Replace (seen) tokens with very low unigram counts in the training data by `<UNK>` and then learn the models as before.

```
Blaardvarks and sharks are neat.
We met in Iguaracy.
I met Zeferino yesterday.
```

becomes:

```
<UNK> and sharks are neat.
We met in <UNK>.
I met <UNK> yesterday.
```

When we encounter an OOV token in a test sentence, replace it by `<UNK>` and estimate the $n$-gram probabilities as before.

- We "forced" `<UNK>` to be in-vocabulary, so there will be no zeroes!

**Ex:**

`I like boogaboos` is rewritten as `I like <UNK>`, for which an estimate is available.



## Relationship to other modules

1. PCFGs as language models
2. Probabilistic PoS tagging (HMM)
3. Large language models
4. Dialogue systems
    i. Generating responses?
    ii. Testing naturalness of responses?





## What you should know or be able to do

- Build n-gram language models from text
- Estimate the probability of text given a PLM
- Compute perplexity
- Explain data sparsity and how it affects language modelling
- Explain the various methods for addressing data sparsity
- Explain what are OOV terms and how to deal with them
- Explain some applications of PLMs
- Explain how to generate text with PLMs